Trabajo Práctico Integrador: Estructuras de Datos, Contrato Equals/HashCode y Complejidad Algorítmica

Asignatura: Desarrollo de Software

Modalidad: en grupo

Criterio de Evaluación y Política de IA: Se permite el uso de herramientas de Inteligencia Artificial (LLMs).Sin embargo, el práctico está diseñado para evaluar el razonamiento crítico, el análisis de estados de memoria y la justificación técnica. Las respuestas genéricas generadas por IA sin prueba de trazabilidad manual o sin responder a los escenarios de borde específicos serán penalizadas.

Criterios Generales de Entrega: Entregar un informe en formato PDF publicado en un repositorio de GitHub. Incluir fragmentos de código fuente en Java y los diagramas/trazas explicativas solicitadas.

Andres Moreno	52055
Tomas Basabe	51910
Agustina Vega	52150
Ignacio Generoso	51990
Daniel Chirino	52571

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria **2**

1. Traza del Estado Interno: Explique paso a paso qué ocurre dentro de la estructura HashSet: 2

2. Explicación del Cajón (Bucket): ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer contains()? 2

3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7? 2

4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad). 2

Consigna 2: Selección y Diseño de Arquitectura de Colecciones **3**

Consigna 3: Optimización Algorítmica (O(n^2) -> O(n)) **4**

1. Refactorización a O(n): Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales (O(n)). 5

2. Análisis de Colisiones: Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información. 6

Consigna 4: Caso Teórico de Borde y Colisiones **6**

1. Si guardamos ambas cadenas en un HashSet<String>, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos hashCode() y equals() internamente para determinar si s2 debe ingresarse o desecharse. 6

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un HashMap en el peor escenario posible (de O(1) a O(n) o O(\log n))? 6

Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

1. Traza del Estado Interno: Explique paso a paso qué ocurre dentro de la estructura HashSet:

- Al ejecutar tarjetasValidas.add(t1).
  Al ejecutar tarjetasValidas.add(t1) el hash calcula el Bucket en la que se va a almacenar la tarjeta utilizando el código de la tarjeta y su nivel de acceso. Posteriormente la guarda en la ubicación que calculó.

- Al ejecutar t1.setNivelAcceso(5).
  Al ejecutar t1.setNivelAcceso(5) se modifica el valor de la variable NivelAcceso en el Bucket original, un error que no se deberia cometer ya que los atributos que pertenecen a el HashCode no deben ser mutables.

- Al ejecutar tarjetasValidas.contains(t1).
  Al ejecutar tarjetasValidas.contains(t1) java calcula la posición del Bucket en el que debería estar guardada la tarjeta con los datos del código y el nivel de acceso. Posteriormente, ejecuta el método equals() para comprobar si la tarjeta se encuentra allí. Devuelve false dado que la tarjeta originalmente fue guardada en otro Bucket y se modificó uno de los atributos que se utilizan para calcular el Hash. Por lo tanto, no la va a encontrar porque no va a ir a buscarla al Bucket correcto.

2. Explicación del Cajón (Bucket): ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer contains()?

El objeto queda guardado en el primer número que calculó el HashCode para los atributos que se instanciaron primero, pertenecientes a este HashCode. Luego, al hacer el contains() Java recalcula con el hashcode la ubicacion, al estar modificado uno de los atributos, dará un nuevo valor de Bucket por lo que el objeto, no se encontrará en ese nuevo Bucket.

3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

Al no poder encontrar los datos que han sido modificados, los mismos quedan perpetuamente en memoria sobrecargandola. Esto es un problema al tener un servidor que opera constantemente ya que se producirá una gran acumulación de datos que no pueden ser borrados por el usuario ni por el recolector de basura de java.

4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

Solución 1: Inmutabilidad
Hago que los atributos utilizados en el Hash sean atributos finales

public final class TarjetaAcceso {

```
    private final String codigo;
    private final int nivelAcceso;
}
```

Solución 2: Cambiar parámetros en Hash y equals
Eliminamos el atributo nivelAcceso de equals y hashCode

```
@Override
public boolean equals(Object o) {
   if (this == o) return true;
   if (o == null || getClass() != o.getClass()) return false;
   TarjetaAcceso tarjeta = (TarjetaAcceso) o;
   return Objects.equals(codigo, tarjeta.codigo);
 }
@Override
public int hashCode() {
  return Objects.hash(codigo);
}
```

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Diseñe el backend de un módulo para una plataforma de E-Commerce. Debe elegir la interfaz y la implementación concreta del Java Collections Framework (java.util) más eficiente para cada una de las siguientes tres funcionalidades:

```
Requerimiento 1: Sistema de Auditoría de Transacciones
Registrar operaciones en el orden exacto en que ocurren. Se requiere insertar miles
de registros por segundo al final de la colección y recorrerlos secuencialmente.

Requerimiento 2: Catálogo de Ofertas Relámpago
Mantener los productos ordenados automáticamente por precio de menor a mayor.
Debe permitir consultar eficientemente el producto más barato y el más caro.

Requerimiento 3: Búsqueda de Sesiones de Usuarios Activos
Buscar instantáneamente la sesión de un usuario dado su Token (String único) entre
500,000 usuarios conectados en simultáneo.
```

Requerimiento	Interfaz Seleccionada	Implementación Concreta	Complejidad Búsqueda	Complejidad Inserción	Justificación Técnica de Elección
Auditoría	List	ArrayList	$O(N)$ lineal / $O(1)$ por índice	$O(1)$ amortizado al final	Un log de auditoría debe admitir

					transacciones duplicadas y conservar orden cronológico. ArrayList ofrece inserción al final en $O(1)$ amortizado, bajo consumo de memoria y máxima velocidad en lecturas secuenciales gracias a la localidad espacial de memoria caché. *(Un Set no sirve porque rechaza duplicados y gasta CPU calculando hashes innecesarios)*.
Ofertas	Set	TreeSet	$O(1)$ extremos / $O(\log N)$ general	$O(\log N)$	Al basarse en un árbol Red-Black, mantiene los productos ordenados automáticamente por precio. Permite consultar de forma inmediata el más barato (`first()`) y el más caro (`last()`) en tiempo constante/logarítmico.
Sesiones	Map	HashMap	$O(1)$ promedio	$O(1)$ promedio	Permite indexar por clave única (Token).

					Calcula el bucket mediante el hash del String y accede al objeto de sesión en tiempo constante $O(1)$ promedio, sin importar que haya 500,000 usuarios activos.

Consigna 3: Optimización Algorítmica (O(n^2) -> O(n))

Un desarrollador escribió el siguiente método para encontrar el primer carácter que no se repite dentro de una cadena de texto (ejemplo: en "barco", el primero es 'b'; en "abarca", el primero es 'b').

```
public static Character primerCaracterNoRepetidoLento(String texto) {
// Algoritmo ineficiente O(n^2)
for (int i = 0; i < texto.length(); i++) {
char c = texto.charAt(i);
boolean duplicado = false;
for (int j = 0; j < texto.length(); j++) {
if (i != j && c == texto.charAt(j)) {
duplicado = true;
break;
}
}
if (!duplicado) return c;
}
return null;
}
```

1. Refactorización a O(n): Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales (O(n)).

Para resolver el problema de forma óptima elegí usar un LinkedHashMap<Character, Integer>. Esta colección nos permite contar las frecuencias en tiempo constante como cualquier HashMap, pero con la ventaja de que mantiene el orden de inserción original, lo que nos deja saber cuál fue el primer carácter no repetido sin tener que recorrer todo otra vez.

El código queda así:

```
import java.util.LinkedHashMap;
import java.util.Map;

public Character primerCaracterNoRepetido(String texto) {
    if (texto == null || texto.isEmpty()) {
        return null;
    }

    Map<Character, Integer> frecuencias = new LinkedHashMap<>();

    // Primer recorrido: contamos las apariciones de cada carácter
    for (int i = 0; i < texto.length(); i++) {
        char c = texto.charAt(i);
        frecuencias.put(c, frecuencias.getOrDefault(c, 0) + 1);
    }

    // Segundo recorrido: buscamos la primera entrada que tenga frecuencia 1
    for (Map.Entry<Character, Integer> entrada : frecuencias.entrySet()) {
        if (entrada.getValue() == 1) {
            return entrada.getKey();
        }
    }

    return null;
}
```

2. Análisis de Colisiones: Si se utiliza un HashMap internamente, explique qué ocurre si dos caracteres distintos generan el mismo hashCode() (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Una colisión de hash ocurre cuando dos claves o caracteres distintos generan el mismo hashCode(), haciendo que Java intente guardarlos en el mismo Bucket dentro de la tabla. Java resuelve esto usando la técnica de encadenamiento (Separate Chaining):

- Cuando ocurre una colisión, Java almacena los elementos en ese Bucket usando una lista enlazada de nodos.
- Primero se calcula la posición del Bucket mediante el hashCode(). Si ya hay un elemento guardado allí, Java ejecuta el método equals() para comparar la clave nueva con las existentes.
- Si equals() devuelve true, se sobreescribe el valor actual porque se trata de la misma clave. Si devuelve false, Java reconoce que es una clave diferente que colisionó en la misma posición y la agrega como un nodo nuevo a la lista.
- Si un Bucket llega a tener 8 o más elementos encadenados, Java convierte esa lista en un árbol rojo-negro para mantener las búsquedas rápidas en $O(\log n)$.

De esta manera nunca se pierde información, ya que el hashCode() solo define la ubicación del Bucket, mientras que el método equals() es el que se encarga de diferenciar exactamente cada elemento.

Consigna 4: Caso Teórico de Borde y Colisiones

1. Si guardamos ambas cadenas en un HashSet<String>, ¿cuántos elementos contendrá la colección? Explique cómo utiliza Java los métodos hashCode() y equals() internamente para determinar si s2 debe ingresarse o desecharse.

La colección final contendrá 2 elementos. Primero el hashCode() localiza el bucket en la memoria donde debe ir el objeto. Al hacer eso se da que FB y Ea dan el número 2236, por lo que son enviadas al mismo bucket y se hace una colisión de hash.

Esto no significa que sean lógicamente iguales, por lo que se utiliza un equals() que determina que son distintos, por lo que el HashSet ingresa a s2 como un elemento válido y no desecha ninguno.

2. ¿Cómo afecta la acumulación de colisiones de hash al rendimiento teórico de un HashMap en el peor escenario posible (de O(1) a O(n) o O(\log n))?

La acumulación de estas colisiones degrada el rendimiento de la estructura porque pasa de una búsqueda ideal de O(1) a uno de O(n) en el peor de los casos. Esto sucede porque los objetos colisionados forman una lista enlazada que se debe revisar elemento por elemento y como la notación siempre contempla el peor escenario , el esfuerzo crecerá en proporción directa a la cantidad de elementos colisionados.